LONDON METROPOLITAN UNIVERSITY

islington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

Level 6 – Data and Web Development

Assessment Type

Semester 2025/26 Autumn

Student Name:Aaishma Manandhar

London Met ID: 23050191

College ID: NP01CP4A230436

Assignment Due Date: March 9th, 2026

Assignment Submission Date: March 9th, 2026

Submitted To: Abhishek Anand

Word Count (Where Required):3799

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submissions and a mark of zero will be awarded.

Table of Contents

1. Introduction 5
1.1BusinessRule 5
2. Textual Analysis 6
2.1 Customer and Ticket 7
2.2 Show and Ticket 7
2.3 Hall and Show 8
2.4 Theatre and Hall 8
2.5 Theatre and Movie 9
2.6 Theatre and Movie 9
2.7 Movie and Show 9
2.8 Ticket and Payment 10
2.9 Customer and Show 10
2.10 Hall and Movie 10
3. ERD from Case Study 11
4. Normalization 12
4.1 UNF 12
4.2 1NF 12
4.3 2nf 13
Final 2nf Relation 16
4.4 3NF 17
5. Integration and Assumption 20
6. Final ERD 22
7. Data Dictionary 23
8. Script: 32
8.1 DDL Script of Data Module 32
9. Create Statement 39
9.1 Create statement of customer Table 39
9.2 Create Statement of Theatre Table 39
9.3 Create Statement of Movie Table 40
9.4 Create Statement of payment Table 40
9.5 Create Statement of Ticket Table 41
9.6 Create Statement of Show Table 42
9.7 Create Statement of Hall Table 42
9.8 Create Statement of Theatre_Movie Table 42
9.9 Create Statement of Hall_Theatre_Movie Table 43
9.10 Create Statement of Show_Hall_Theatre_Movie Table 44

9.11 Create Statement of Ticket_Customer Table 44
9. Insert Statement 45
9.1 Insert Statement of Customer table 45
9.2 Insert Statement of Theatre table 45
9.3 Insert Statement of Movie table 46
9.4 Insert Statement of Payment table 46
9.5 Insert Statement of Ticket table 47
9.6 Insert Statement of Show table 47
9.7 Insert Statement of Hall table 48
9.8 Insert Statement of Theatre_Movie table 48
9.9 Insert Statement of Hall_Theatre_Movie table 49
9.10 Insert Statement of Show_Hall_Theatre_Movie table 49
9.11 Insert Statement of Ticket_Customer table 50
10. Select Statement 50
10.1 Select Statement of Customer table 50
10.2 Select Statement of Theatre table 51
10.3 Select Statement of Movie table 51
10.4 Select Statement of Payment table 52
10.5 Select Statement of Ticket table 52
10.6 Select Statement of Show table 53
10.7 Select Statement of Hall table 53
10.8 Select Statement of Theatre_Movie table 54
10.9 Select Statement of Hall_Theatre_Movie table 54
10.10 Select Statement of Show_Hall_Theatre_Movie table 55
10.11 Select Statement of Ticket_Customer table 55
11. Forms 56
11.1 Dashboard page 56
11.2 Complex form and queries 56
11.2.1 SQL Queries 56
11.2.2 Complex Form 59
11.3 Simple Form 60
11.3.1 Customer.aspx 60
11.3.2Movie.aspx 61
11.3.3 Theatre.aspx 61
11.3.4 Hall.aspx 61
11.3.5 Show.aspx 62
11.3.6 Ticket.aspx 62

11.3.7 Payment.aspx 62
15. Conclusion 63

Table of Figure

Figure 1: Initial ERD 11
Figure 2: Final ERD 22
Figure 3:DDL SS 1 ..... **Error! Bookmark not defined.**
Figure 4: DD SS 2 ..... **Error! Bookmark not defined.**
Figure 5: DDL 3 ..... **Error! Bookmark not defined.**
Figure 6: DDL SS 4 ..... **Error! Bookmark not defined.**

1. Introduction

Kumari Cinemas is intending to design an online movie ticketing system to enable it to effectively run its activities, comprising of customers, films, theatres, halls, performances, and booking of tickets. The manual system of counter booking of tickets is time consuming, prone to errors and can make the customer inconvenienced. As the digital platforms grow, there exists a substantial demand of a system that will not only enable customers book tickets online but also allow the cinema management to monitor movie shows, halls capacities and sales of tickets efficiently.

The aim of this system is to enhance operational efficiency, minimize the human error and offer a smooth customer experience as well as a smooth experience to the cinema staff. The site will enable the customers to watch the available movies, check the timing of the shows, book seats, pay, and get confirmation in real time. To the management, it allows the management to trace theatre halls, movie schedules, ticket sales and customer information in a systematic and orderly fashion.

This system is developed by designing a relational database that retains all the information related to this database in a normalized format. The normalization process will ensure that there is a low amount of data redundancy, that there are no update anomalies and that have a scalable solution which will be able to support new features in the future. Through this system, Kumari Cinemas will increase customer satisfaction, streamline theatre operation, and simplify booking process.

1.1BusinessRule

Kumari Cinemas plans to develop an online movie ticketing system to manage customers, movies, theatres, halls, shows, and ticket booking effectively. To operate

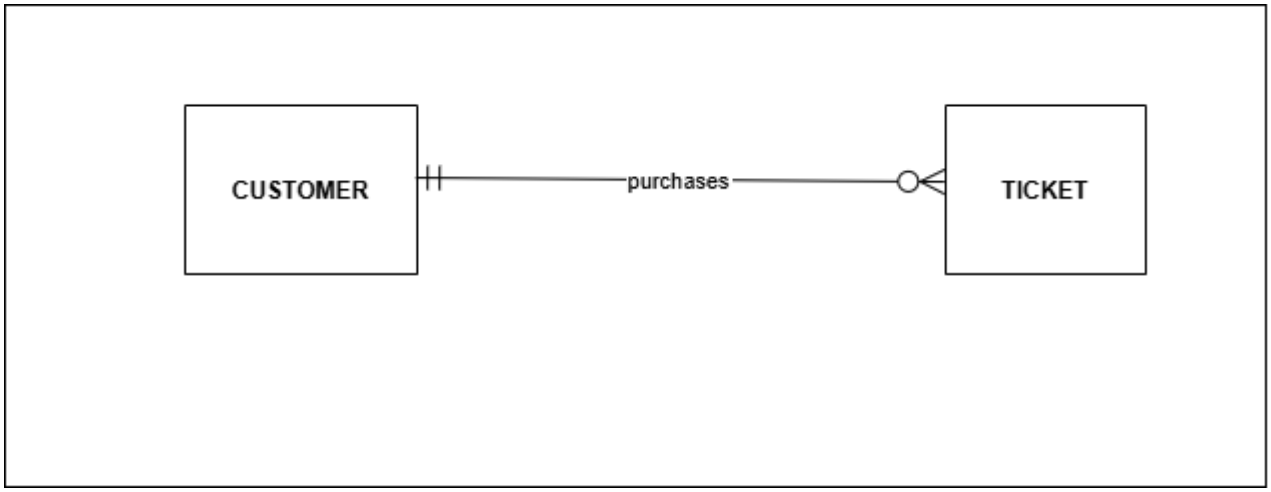
the Movie Ticketing System successfully, the following business rules have be identified:

- One Customer can purchase zero or many Tickets. Each Ticket must be associated with exactly one Customer.
- One Ticket is booked for exactly one Show. Each Show can have one or many Tickets.
- One Show is screened in exactly one Hall. One Hall can host zero or many Shows.
- One Hall belongs to exactly one Theatre. One Theatre can have one or many Halls.
- One Movie can be shown in one or many Theatres. One Theatre can screen one or many Movies.
- One Show features exactly one Movie. One Movie can have one or many Shows.
- One Hall can host zero or many Shows over time. Each Show is held in exactly one Hall.
- One Ticket generates exactly one Payment record. Each Payment corresponds to exactly one Ticket.
- One Customer can have zero or many Tickets/Bookings. Each Ticket is associated with exactly one Customer.

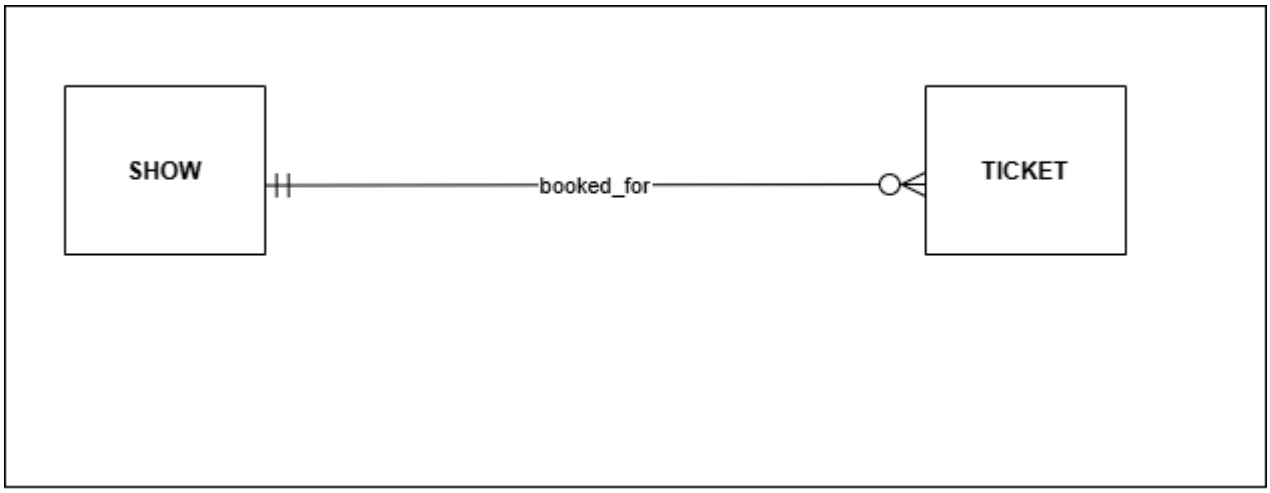
2. Textual Analysis

The textual analysis below describes the relationship between each pair of entities identified in the Movie Ticketing System case study. Each figure documents the cardinality and participation constraints of the relationship.

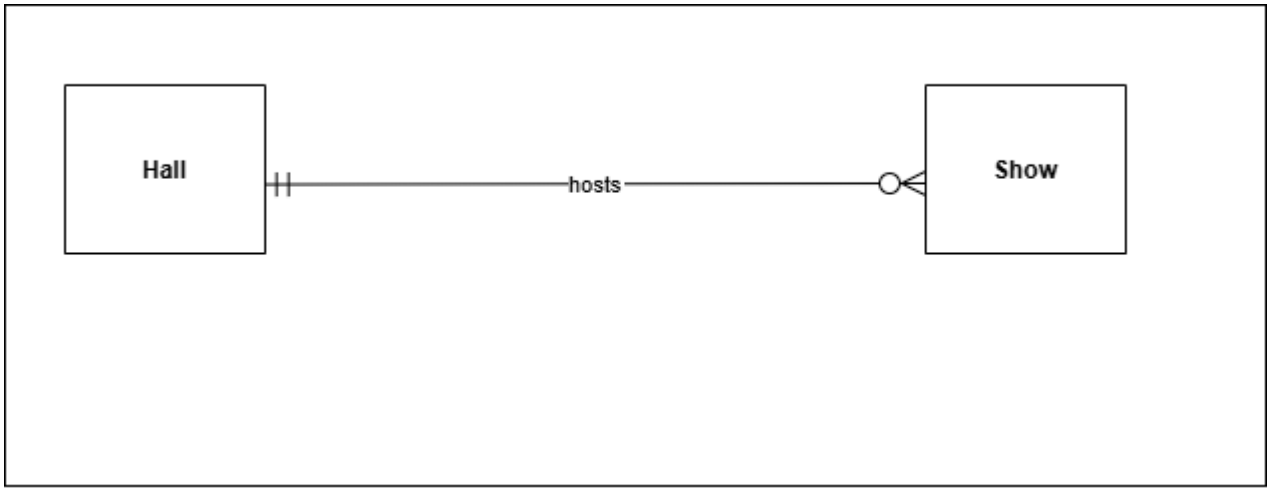
2.1 Customer and Ticket



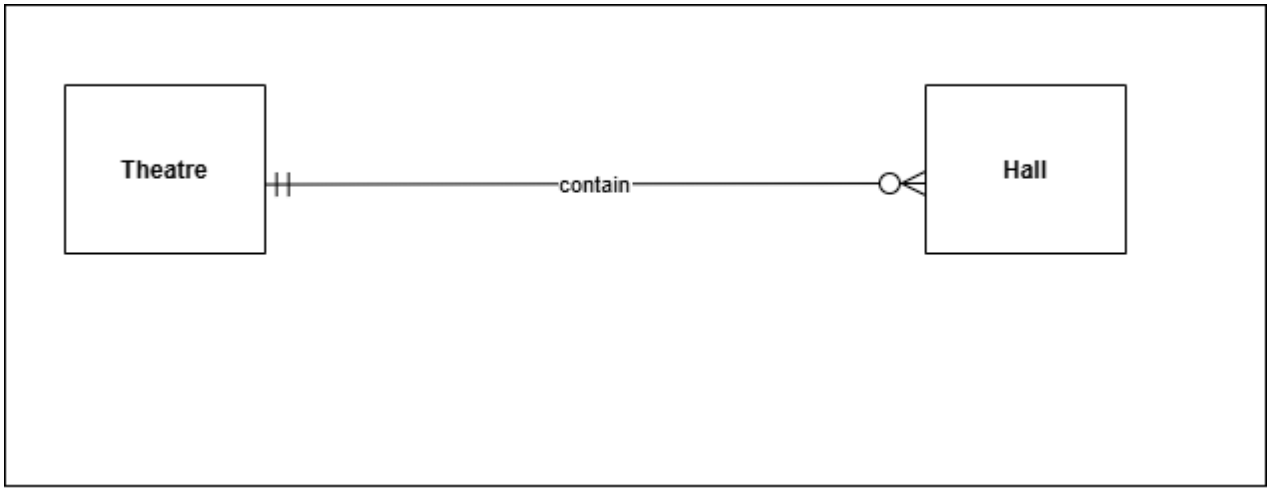
2.2 Show and Ticket



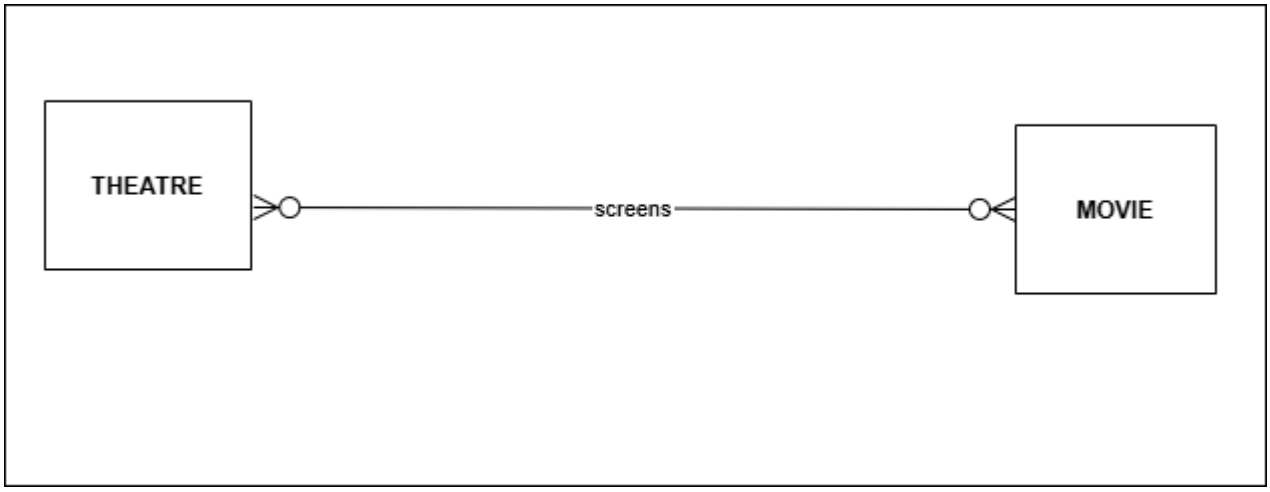
2.3 Hall and Show



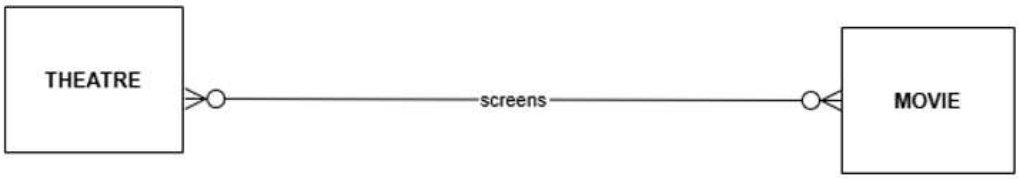
2.4 Theatre and Hall



2.5 Theatre and Movie



2.6 Theatre and Movie



2.7 Movie and Show

2.8 Ticket and Payment

2.9 Customer and Show

2.10 Hall and Movie

3. ERD from Case Study

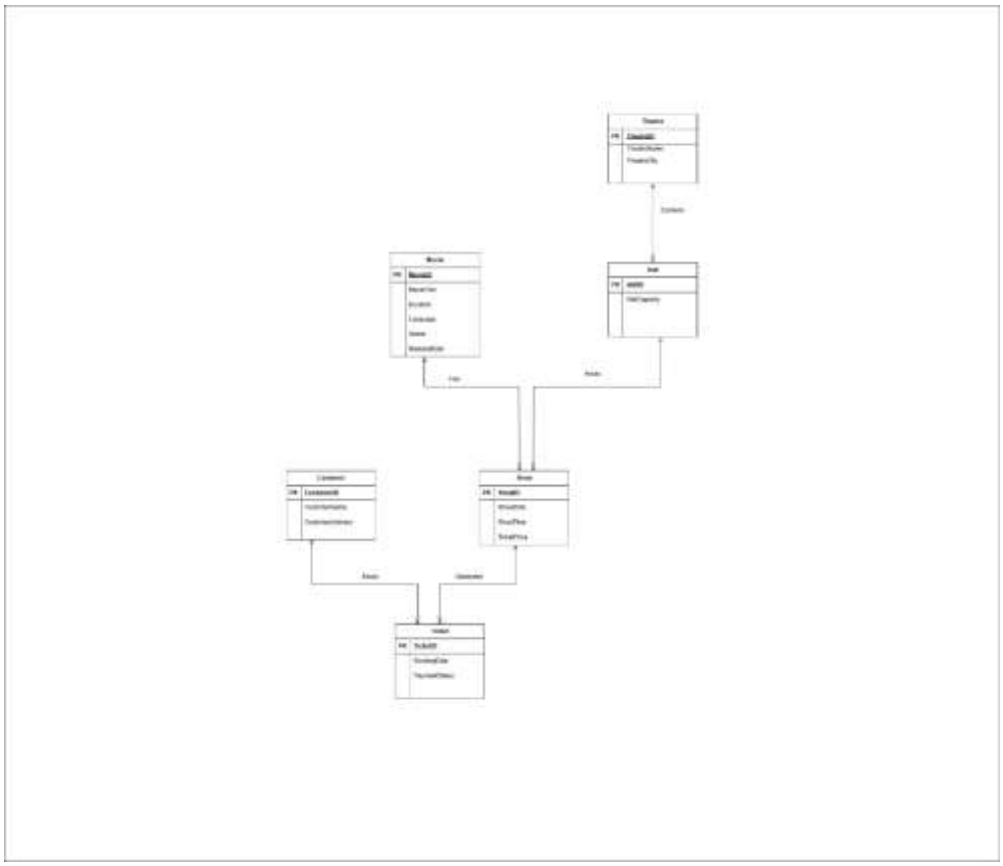


Figure 1: Initial ERD

4. Normalization

4.1 UNF

An unnormalized or UNF frelation is one that does not in any way undergo the process of normalization because it exhibits many anamolies. This form shows all entities and attributes in one relation, with repeating groups using curly brackets { }.

MovieTicketing

(**CustomerID,** CustomerName, CustomerAddress, {MovieID, MovieTitle, Duration, Language, Genre, {TheatreID, TheatreName, TheatreCity, {HallID, HallName, HallCapacity, {ShowID, ShowDate, ShowTime, {TicketID, NumberOfSeat, TicketPrice BookingDate, Payment}}}})

In this UNF, we have identified uniques primary key which is CustomerID. And it is underlined. Also, repeating groups of data is enclosed in curly bracket.

4.2 1NF

In 1NF, Separating non-repeating groups and repeating groups.

Customer-1 : (**<u>CustomerID</u>**, CustomerName, CustomerAddress)

Movie-1: (**<u>MovieID</u>**, MovieTitle, Duration, Language, Genre, ReleaseDate)

Theatre: (**<u>TheatreID</u>**, TheatreName, TheatreCity)

Hall-1(**<u>HallID</u>**,HallName, HallCapacity)

Show-1: (**<u>ShowID</u>**, ShowTime, ShowDate)

Ticket-1: (**<u>TicketID</u>**, BookingDate,NumberOfSeat, Payment)

4.3 2nf

A relation is said to be in Second Normal Form (2NF) if:

1. It is already in First Normal Form (1NF), and
2. There are no partial dependencies, i.e., every non-key attribute is fully functionally dependent on the entire primary key, not just a part of it.

Partial dependency occurs when a table has a **composite primary key**, and a non-key attribute depends on **only one part** of that composite key.

Customer

1nf:

Customer 1: (**<u>CustomerID</u>**, CustomerName, CustomerAddress)

Functional Dependency:

- **CustomerID**→ CustomerName, CustomerAddress

Partial Dependency Check:

- Single key → Cannot have partial dependency

Final 2nf Table:

Customer-2(CustomerID, CustomerName, CustomerAddress)

Movie Table

1Nf:

Movie-1 (**<u>MovieID</u>**, MovieTitle, Duration, Language, Genre, ReleaseDate)

Functional Dependency:

- **MovieID**→ MovieTitle, Duration, Language, Genre, ReleaseDate

Partial Dependency Check:

- Single key → Cannot have partial dependency

Final 2NF Table:

Movie-2(MovieID, MovieTitle, Duration, Language, Genre, ReleaseDate)

TheatreTable:

1NF:

Theatre: (**TheatreID**, TheatreName, TheatreCity, **MovieID***)

Functional Dependency:

- **TheatreID** →TheatreName, Theatre City

Partial Dependency Check:

TheatreName and TheatreCity is depends only on TheatreID Not MovieID. So it became Partial dependency.

Final 2NF Table:

Theatre-2: (**TheatreID,** TheatreName, TheatreCity)

Theatre_Movie-2: (**TheatreID*, MovieID***)

Hall Table:

1NF:

Hall-1 (**TheatreID*,MovieID*, HallID**, HallName, HallCapacity)

Functional Dependency:

HallID → HallName, HallCapacity

Partial Dependency Check:

- HallName and HallCapacity depends only on HallID, not ThreatreID and MovieID → Partial dependency exist

Final 2NF Table:

- **Hall-2**: (**HallID,** HallName, HallCapacity)
- **Hall_Theatre_Movie-2:** (**TheatreID*, MovieID*, HallID***)

Show Table

1NF:

Show-1(**ShowID,** ShowTime, ShowDate, **HallID*,MovieID***,**TheatreID***)

Functional Dependency:

- **ShowID**→ ShowTime, ShowDate

Partial Dependency Check:

- ShowTime, ShowDate depend only on ShowID, not on HallID , MovieID and TheatreID → partial dependency exits.

Final 2NF Table:

- **Show-2**: (**ShowID,** ShowTime, ShowDate)
- **Show_Hall_Theatre_Movie-2:** (**ShowID*,TheatreID*, MovieID*, HallID***)

Ticket Table

1NF:

Ticket-1 (TicketID, NumberOfSeat, BookingDate, Payment, **CustomerID***)

Functional Dependency:

- **TicketID** → BookingDate, Payment, NumberOfSeat

Partial Dependency Check:

- NumberOfSeat, BookingDate and Payment depend only on TicketID Not on CustomerID → Partial dependency exists.

Final 2NF Table:

- **Ticket-2**: (**TicketID**, NumberOfSeat, BookingDate, Payment)
- **Ticket_Customer-2:** (**TicketID*, CustomerID*)**

Final 2nf Relation

Customer-2: (**CustomerID**, CustomerName, CustomerAddress)

Movie-2: (**MovieID**, MovieTitle, Duration, Language, Genre, ReleaseDate)

Threatre-2: (**TheatreID**, TheatreName, TheatreCity)

Hall-2: (**HallID,** HallName, HallCapacity)

Show-2: (**ShowID,** ShowTime, ShowDate)

Ticket-2: (**TicketID**, NumberOfSeat, BookingDate, Payment)

Theatre_Movie-2: (**TheatreID*, MovieID***)

Hall_Theatre_Movie-2: (**TheatreID*, MovieID*, HallID***)

Show_Hall_Theatre_Movie-2: (**ShowID*,TheatreID*, MovieID*, HallID***)

Ticket_Customer-2: (**TicketID*, CustomerID*)**

4.4 3NF

A relation is in **3NF** if:

1. It is in 2NF**.**
2. No transitive dependencies exist → non-key attributes should depend only on the primary key, not on other non-key attributes.

Transitive dependencies create redundancy. In 3NF, we remove them by creating separate tables.

Customer Table:

2NF:

Customer-2 (**CustomerID**, CustomerName, CustomerAddress)

Check for transitive dependencies:

- CustomerName and CustomerAddress depend directly on CustomerID → no transitive dependency

Final 3NF Table:

Customer-3:(**CustomerID,** CustomerName, CustomerAddress)

MovieTable:

2NF:

Movie-2(**MovieID**, MovieTitle, Duration, Language, Genre, ReleaseDate)

Check for transitive dependencies:

- All attributes depend directly on MovieID → no transitive dependency

3NFTable:

Movie-3: (<u>MovieID</u>, MovieTitle, Duration, Language, Genre, ReleaseDate**)**

Theatre Table:

2NF: Theatre-2 (**<u>TheatreID,</u>** TheatreName, TheatreCity)

Check for transitive dependencies:

- TheatreCity and TheatreName depend on TheatreID → No **transitive dependency exists**

Final 3NF

1. Theatre-3(**<u>TheatreID</u>**, **TheatreName, TheatreCity)**

Hall Table

2NF: Hall-2(HallID, HallCapacity)

ThratreHall-2: (**TheatreID*, HallID*)**

Check for transitive dependencies:

- HallCapacity depends directly on HallID → no transitive dependency
- TheatreHall links TheatreID to HallID →no transitive depdendency

3NF Tables:

- Show-3 (**<u>ShowID</u>**, ShowTime, ShowDate, TicketPrice)
- HallMovieShow-3 (**HallID*, MovieID*, ShowID*)**

Ticket Table:

2NF: Ticket-2(TicketID, NumberOfSeat, BookingDate, Payment)

ShowCustomerTicket-2: (**ShowID*, CustomerID*, TicketID*)**

Check for transitive dependencies:

- PaymentStatus depend only BookingDate not on TicketID → So,transitive dependency Exist

Final 3NF Tables:

- Ticket-3: (**TicketID**, BookingDate*, NumberOfSeat)
- **Payment-3**: (**BookingDate**, Payment)

Final 3NF Tables:

Customer-3: (**CustomerID**, CustomerName, CustomerAddress)

Movie-3: (**MovieID**, MovieTitle, Duration, Language, Genre, ReleaseDate)

Threatre-3: (**TheatreID**, TheatreName, TheatreCity)

Hall-3: (**HallID,** HallName, HallCapacity)

Show-3: (**ShowID,** ShowTime, ShowDate)

Theatre_Movie-3: (**TheatreID*, MovieID***)

Hall_Theatre_Movie-3: (**TheatreID*, MovieID*, HallID***)

Show_Hall_Theatre_Movie-3: (**ShowID*,TheatreID*, MovieID*, HallID***)

Ticket_Customer-3: (**TicketID*, CustomerID*)**

Ticket-3: (**TicketID**, BookingDate*, NumberOfSeat)

Payment-3: (**BookingDate**, Payment)

5. Integration and Assumption

After completing the normalization process up to Third Normal Form (3NF), all the resulting entities were integrated to form the final database schema for the Movie Ticketing System. During the integration process, the relationships between entities were carefully analyzed and many-to-many relationships were resolved using associative entities such as Theatre_Movie, Hall_Theatre_Movie, Show_Hall_Theatre_Movie, and Ticket_Customer.

Each entity is connected through primary keys and foreign keys to maintain referential integrity and avoid redundancy. The final integrated schema ensures that the database structure supports efficient storage, retrieval, and management of movie ticketing information.

Assumptions

The following assumptions were made while integrating the entities:

1. Each customer can book multiple tickets, but each ticket is associated with only one customer.
2. A movie can be shown in multiple theatres, and a theatre can screen multiple movies. Therefore, the associative entity Theatre_Movie is used to resolve this many-to-many relationship.
3. Each theatre can contain multiple halls, but each hall belongs to only one theatre.

4. A hall can host multiple shows, but each show takes place in only one hall at a specific date and time.
5. A movie can have multiple shows in different halls and theatres.
6. Each ticket is generated for a specific show and customer booking.
7. Payment information is recorded separately to manage booking payment details.
8. Each show is uniquely identified by ShowID, and the schedule is determined using ShowDate and ShowTime.

Final Integrated Entities with Keys

Customer
(CustomerID PK, CustomerName, CustomerAddress)

Movie
(MovieID PK, MovieTitle, Duration, Language, Genre, ReleaseDate)

Theatre
(TheatreID PK, TheatreName, TheatreCity)

Hall
(HallID PK, HallName, HallCapacity)

Show
(ShowID PK, ShowTime, ShowDate)

Ticket
(TicketID PK, NumberOfSeat, BookingDate FK)

Payment
(BookingDate PK, Payment)

Associative Entities

Theatre_Movie
(TheatreID PK/FK, MovieID PK/FK)

Hall_Theatre_Movie

(TheatreID PK/FK, MovieID PK/FK, HallID PK/FK)

Show_Hall_Theatre_Movie

(ShowID PK/FK, TheatreID PK/FK, MovieID PK/FK, HallID PK/FK)

Ticket_Customer

(TicketID PK/FK, CustomerID PK/FK)

6. Final ERD

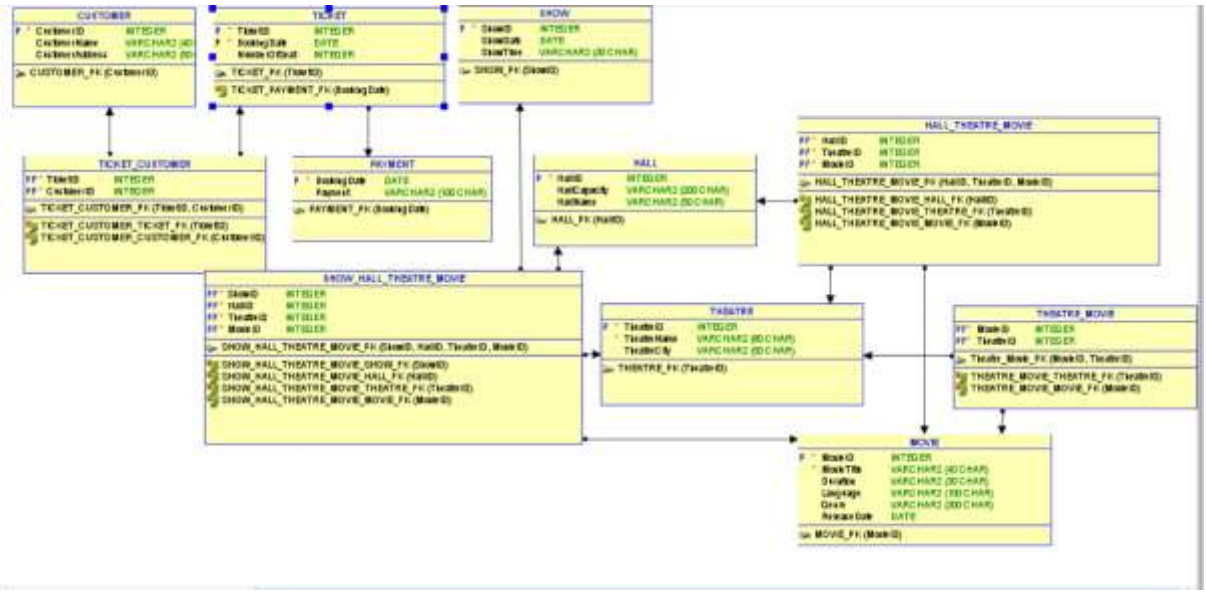


Figure 2: Final ERD

7. Data Dictionary

The data dictionary contains information of all the tables utilized in the database. It defines every attribute such as its data type, size, constraints, references and intention in the system. This assists in the interpretation of the structure and relations amongst various entities of the database. (GeeksforGeeks, 23 July 2025).

Table: Customer

The **Customer** table stores information about users who book movie tickets. Each customer is uniquely identified by a CustomerID and their personal details such as name and address are stored in this table. This table helps track which customer made a ticket booking.

Column Name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example Data
CustomerID	INT	10	Primary Key	-	-	To uniquely identify each customer	C101
CustomerName	VARCHAR	100	Not Null	-	-	To store the	Ram Shrestha

						customer name	
CustomerAddress	Varchar	255	Not Null	-	-	To store the address of the customer	Kathmandu

Table: Movie

The **Movie** table stores details about all movies available in the theatre system. It includes information such as movie title, duration, language, genre, and release date. Each movie is uniquely identified using the MovieID.

Column Name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example Data
MovieID	INT	10	Primary Key	-	-	To uniquely identify each movie	M101
MovieTitle	VARCHAR	150	NOT NULL	-	-	To store the movie title	Avatar
Duration	INT	5	NOT NULL	-	-	To store movie duration in minutes	150

Language	VARCHAR	50	NOT NULL	-	-	To store movie language	English
Genre	VARCHAR	50	NOT NULL	-	-	To store movie category	Action
ReleaseDate	DATE	-	NOT NULL	-	-	To store movie category	2023-12-15

Table: Theatre

The **Theatre** table contains information about theatres where movies are shown. It stores the theatre name and the city where it is located. Each theatre is uniquely identified using TheatreID.

Column Name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example Data
TheatreID	INT	10	Primary Key	-	-	To uniquely identify each theatre	A101
TheatreName	VARCHAR	100	NOT NULL	-	-	To store theatre name	QFX Cinemas
TheatreCity	VARCHAR	100	NOT NULL	-	-	To store theatre city	Kathmandu

Table: Hall

The **Hall** table stores information about halls inside each theatre. Each hall has a specific name and seating capacity that determines how many viewers it can accommodate. HallID uniquely identifies each hall.

Column Name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example Data
HallID	INT	10	Primary Key	-	-	To uniquely identify each hall	H01
HallName	VARCHAR	50	NOT NULL	-	-	To store hall name or number	200
HallCapacity	INT	5	NOT NULL	-	-	To store maximum seating capacity	200

Table: Show

The **Show** table contains details about movie show schedules. It records the date and time when a particular show is conducted. Each show is uniquely identified by ShowID.

Column Name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example Data
ShowID	INT	10	Primary Key	-	-	To uniquely	S101

						identify each hall	
ShowTime	TIME	-	NOT NULL	-	-	To store show starting time	18:00
ShowDate	DATE	-	NOT NULL	-	-	To store show date	2026-03-10

Table: Theatre_Movie

The **Theatre_Movie** table represents the relationship between theatres and movies. It indicates which movies are available in which theatres. This table helps manage many-to-many relationships between theatres and movies.

Column Name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example Data
TheatreID	INT	10	Foreign Key	Theatre	TheatreID	To identify theatre where movie is shown	A101
MovieID	INT	10	Foreign Key	Movie	MovieID	To identify movie shown in theatre	M101

Table: Hall_Theatre_Movie

The **Hall_Theatre_Movie** table shows which hall in a specific theatre is used to screen a particular movie. This helps manage movie allocation within theatres.

Column Name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example Data
TheatreID	INT	10	Foreign Key	Theatre	TheatreID	To identify theatre	A101
MovieID	INT	10	Foreign Key	Movie	MovieID	To identify movie	M101
HallID	INT	10	Foreign Key	Hall	HallID	To identify hall where movie plays	H01

Table: Show_Hall_Theatre_Movie

The **Show_Hall_Theatre_Movie** table stores information about which show is conducted in which hall for a particular movie in a theatre. It helps link show schedules with movie locations.

Column Name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example Data
ShowID	INT	10	Foreign Key	Show	ShowID	To identify specific show	S101
TheatreID	INT	10	Foreign Key	Theatre	TheatreID	To identify theatre	A101

MovieID	INT	10	Foreign Key	Movie	MovieID	To identify movie	M101
HallID	INT	10	Foreign Key	Hall	HallID	To identify hall where movie plays	H01

Table: Ticket

The **Ticket** table stores booking information for movie tickets. It records the number of seats booked and the booking date. Each ticket booking is uniquely identified by TicketID.

Column Name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example Data
TicketID	INT	10	Primary Key	-	-	To uniquely identify each ticket	TK101
BookingDate	DATE	-	Foreign Key	Payment	BookingDate	To store ticket booking date	2026-03-05
NumberOfSeat	INT	3	NOT NULL	-	-	To store number of seats booked	3

Table: Ticket_Customer

The **Ticket_Customer** table represents the relationship between tickets and customers. It identifies which customer has booked which ticket.

Column Name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example Data
TicketID	INT	10	Foreign Key	Ticket	TicketID	To identify booked ticket	TK101
CustomerID	INT	10	Foreign Key	Customer	CustomerID	To identify customer who booked ticket	C101

Table: Payment

The **Payment** table stores payment details related to ticket bookings. It records the payment amount and booking date associated with the transaction.

Column Name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example Data

BookingDate	DATE	-	Primary Key	-	-	To identify payment date	2026-03-05
Payment	DECIMAL	10,2	NOT NULL	-	-	To store payment amount	1500.00

8. Script:

8.1 DDL Script of Data Module

```
CREATE TABLE customer (
    customerid      INTEGER NOT NULL,
    customername    VARCHAR2(40 CHAR),
    customeraddress VARCHAR2(50 CHAR)
);

ALTER TABLE customer ADD CONSTRAINT customer_pk PRIMARY KEY ( customerid );

CREATE TABLE theatre (
    theatreid   INTEGER NOT NULL,
    theatrename VARCHAR2(80 CHAR) NOT NULL,
    theatrecity VARCHAR2(50 CHAR)
);

ALTER TABLE theatre ADD CONSTRAINT theatre_pk PRIMARY KEY ( theatreid );

CREATE TABLE movie (
    movieid     INTEGER NOT NULL,
    movietitle  VARCHAR2(40 CHAR) NOT NULL,
```

```
    duration    VARCHAR2(30 CHAR),
    language    VARCHAR2(100 CHAR),
    genre       VARCHAR2(300 CHAR),
    releasedate DATE
);

ALTER TABLE movie ADD CONSTRAINT movie_pk PRIMARY KEY ( movieid );
CREATE TABLE hall (
    hallid       INTEGER NOT NULL,
    hallcapacity VARCHAR2(200 CHAR),
    hallname     VARCHAR2(50 CHAR)
);

ALTER TABLE hall ADD CONSTRAINT hall_pk PRIMARY KEY ( hallid );

CREATE TABLE show (
    showid   INTEGER NOT NULL,
    showdate DATE,
    showtime VARCHAR2(30 CHAR)
);

ALTER TABLE show ADD CONSTRAINT show_pk PRIMARY KEY ( showid );
```

```
CREATE TABLE ticket (
    ticketid     INTEGER NOT NULL,
    bookingdate  DATE NOT NULL,
    numberofseat INTEGER
);

ALTER TABLE ticket ADD CONSTRAINT ticket_pk PRIMARY KEY ( ticketid );

ALTER TABLE ticket
    ADD CONSTRAINT ticket_payment_fk FOREIGN KEY ( bookingdate )
        REFERENCES payment ( bookingdate );

CREATE TABLE ticket_customer (
    ticketid   INTEGER NOT NULL,
    customerid INTEGER NOT NULL
);

ALTER TABLE ticket_customer ADD CONSTRAINT ticket_customer_pk PRIMARY
KEY ( ticketid,
                                                    customerid );

ALTER TABLE ticket_customer
    ADD CONSTRAINT ticket_customer_customer_fk FOREIGN KEY ( customerid )
```

```
        REFERENCES customer ( customerid );


ALTER TABLE ticket_customer
    ADD CONSTRAINT ticket_customer_ticket_fk FOREIGN KEY ( ticketid )
        REFERENCES ticket ( ticketid );


CREATE TABLE show_hall_theatre_movie (
    showid    INTEGER NOT NULL,
    hallid    INTEGER NOT NULL,
    theatreid INTEGER NOT NULL,
    movieid   INTEGER NOT NULL
);


ALTER TABLE show_hall_theatre_movie
    ADD CONSTRAINT shtm_pk PRIMARY KEY ( showid,
                                         hallid,
                                         theatreid,
                                         movieid );


ALTER TABLE show_hall_theatre_movie
    ADD CONSTRAINT s_h_t_m_hl_fk FOREIGN KEY ( hallid )
        REFERENCES hall ( hallid );
```

```
ALTER TABLE show_hall_theatre_movie
    ADD CONSTRAINT s_h_t_m_m_fk FOREIGN KEY ( movieid )
        REFERENCES movie ( movieid );

ALTER TABLE show_hall_theatre_movie
    ADD CONSTRAINT s_h_t_m_s_fk FOREIGN KEY ( showid )
        REFERENCES show ( showid );

ALTER TABLE show_hall_theatre_movie
    ADD CONSTRAINT s_h_t_m_t_fk FOREIGN KEY ( theatreid )
        REFERENCES theatre ( theatreid );

CREATE TABLE hall_theatre_movie (
    hallid    INTEGER NOT NULL,
    theatreid INTEGER NOT NULL,
    movieid   INTEGER NOT NULL
);

ALTER TABLE hall_theatre_movie
    ADD CONSTRAINT hall_theatre_movie_pk PRIMARY KEY ( hallid,
                                                       theatreid,
                                                       movieid );
```

```
ALTER TABLE hall_theatre_movie
    ADD CONSTRAINT hall_theatre_movie_hall_fk FOREIGN KEY ( hallid )
        REFERENCES hall ( hallid );

ALTER TABLE hall_theatre_movie
    ADD CONSTRAINT hall_theatre_movie_movie_fk FOREIGN KEY ( movieid )
        REFERENCES movie ( movieid );

ALTER TABLE hall_theatre_movie
    ADD CONSTRAINT hall_theatre_movie_theatre_fk FOREIGN KEY ( theatreid )
        REFERENCES theatre ( theatreid );
CREATE TABLE theatre_movie (
    movieid   INTEGER NOT NULL,
    theatreid INTEGER NOT NULL
);

ALTER TABLE theatre_movie ADD CONSTRAINT theatre_movie_pk PRIMARY KEY
( movieid,
                                                    theatreid );
ALTER TABLE theatre_movie
    ADD CONSTRAINT theatre_movie_movie_fk FOREIGN KEY ( movieid )
        REFERENCES movie ( movieid );
```

```
ALTER TABLE theatre_movie

    ADD CONSTRAINT theatre_movie_theatre_fk FOREIGN KEY ( theatreid )

        REFERENCES theatre ( theatreid );


CREATE TABLE payment (

    bookingdate DATE NOT NULL,

    payment     VARCHAR2(100 CHAR)

);

ALTER TABLE payment ADD CONSTRAINT payment_pk PRIMARY KEY ( bookingdate );
```

9. Create Statement

9.1 Create statement of customer Table

```
CREATE TABLE customer (
    customerid      INTEGER NOT NULL,
    customername    VARCHAR2(40 CHAR),
    customeraddress VARCHAR2(50 CHAR)
);

ALTER TABLE customer ADD CONSTRAINT customer_pk PRIMARY KEY ( customerid );
```

Script Output

Task completed in 0.097 seconds

```
Table CUSTOMER created.

Table CUSTOMER altered.
```

9.2 Create Statement of Theatre Table

```
CREATE TABLE theatre (
    theatreid   INTEGER NOT NULL,
    theatrename VARCHAR2(80 CHAR) NOT NULL,
    theatrecity VARCHAR2(50 CHAR)
);

ALTER TABLE theatre ADD CONSTRAINT theatre_pk PRIMARY KEY ( theatreid );
```

Script Output

Task completed in 0.049 seconds

```
Table CUSTOMER created.

Table CUSTOMER altered.

Table THEATRE created.

Table THEATRE altered.
```

9.3 Create Statement of Movie Table

```
CREATE TABLE movie (
    movieid     INTEGER NOT NULL,
    movietitle  VARCHAR2(40 CHAR) NOT NULL,
    duration    VARCHAR2(30 CHAR),
    language    VARCHAR2(100 CHAR),
    genre       VARCHAR2(300 CHAR),
    releasedate DATE
);
ALTER TABLE movie ADD CONSTRAINT movie_pk PRIMARY KEY ( movieid );
```

Script Output

Task completed in 0.039 seconds

```
Table THEATRE created.

Table THEATRE altered.

Table MOVIE created.

Table MOVIE altered.
```

9.4 Create Statement of payment Table

```
CREATE TABLE payment (
    bookingdate DATE NOT NULL,
    payment     VARCHAR2(100 CHAR)
);

ALTER TABLE payment ADD CONSTRAINT payment_pk PRIMARY KEY ( bookingdate );



CREATE TABLE ticket (
```

Script Output

Task completed in 0.054 seconds

```
*Action:

Table TICKET altered.


Table SHOW created.


Table PAYMENT created.


Table PAYMENT altered.
```

9.5 Create Statement of Ticket Table

```
CREATE TABLE ticket (
    ticketid     INTEGER NOT NULL,
    bookingdate  DATE NOT NULL,
    numberofseat INTEGER
);
ALTER TABLE ticket ADD CONSTRAINT ticket_pk PRIMARY KEY ( ticketid );

ALTER TABLE ticket
    ADD CONSTRAINT ticket_payment_fk FOREIGN KEY ( bookingdate )
        REFERENCES payment ( bookingdate );
```

Script Output

Task completed in 0.057 seconds

```
    bookingdate DATE NOT NULL,
    payment     VARCHAR2(100 CHAR)
)
Error report -
ORA-00955: name is already used by an existing object
00955. 00000 -  "name is already used by an existing object"
*Cause:
*Action:

Table TICKET altered.
```

9.6 Create Statement of Show Table

```
CREATE TABLE show (
    showid   INTEGER NOT NULL,
    showdate DATE,
    showtime VARCHAR2(30 CHAR)
);
ALTER TABLE show ADD CONSTRAINT show_pk PRIMARY KEY ( showid );
```

Script Output

Task completed in 0.043 seconds

```
Error report -
ORA-00942: table or view does not exist
00942. 00000 -  "table or view does not exist"
*Cause:
*Action:

Table TICKET altered.


Table SHOW created.
```

9.7 Create Statement of Hall Table

```
CREATE TABLE hall (
    hallid       INTEGER NOT NULL,
    hallcapacity VARCHAR2(200 CHAR),
    hallname     VARCHAR2(50 CHAR)
);
ALTER TABLE hall ADD CONSTRAINT hall_pk PRIMARY KEY ( hallid );
```

Script Output

Task completed in 0.032 seconds

```
*Cause:
*Action:

Table TICKET altered.


Table HALL created.


Table HALL altered.
```

9.8 Create Statement of Theatre_Movie Table

```sql
CREATE TABLE theatre_movie (
    movieid   INTEGER NOT NULL,
    theatreid INTEGER NOT NULL
);
ALTER TABLE theatre_movie ADD CONSTRAINT theatre_movie_pk PRIMARY KEY ( movieid,
                                                                        theatreid );

ALTER TABLE theatre_movie
    ADD CONSTRAINT theatre_movie_movie_fk FOREIGN KEY ( movieid )
        REFERENCES movie ( movieid );

ALTER TABLE theatre_movie
```

Script Output

Task completed in 0.046 seconds

```
Table HALL created.

Table HALL altered.

Table THEATRE_MOVIE created.

Table THEATRE_MOVIE altered.
```

9.9 Create Statement of Hall_Theatre_Movie Table

```sql
CREATE TABLE hall_theatre_movie (
    hallid    INTEGER NOT NULL,
    theatreid INTEGER NOT NULL,
    movieid   INTEGER NOT NULL
);
ALTER TABLE hall_theatre_movie
    ADD CONSTRAINT hall_theatre_movie_pk PRIMARY KEY ( hallid,
                                                       theatreid,
                                                       movieid );
```

Script Output

Task completed in 0.055 seconds

```
Table THEATRE_MOVIE altered.

Table HALL_THEATRE_MOVIE created.

Table HALL_THEATRE_MOVIE altered.

Table HALL_THEATRE_MOVIE altered.
```

9.10 Create Statement of Show_Hall_Theatre_Movie Table

```
CREATE TABLE show_hall_theatre_movie (
  showid    INTEGER NOT NULL,
  hallid    INTEGER NOT NULL,
  theatreid INTEGER NOT NULL,
  movieid   INTEGER NOT NULL
);
ALTER TABLE show_hall_theatre_movie
    ADD CONSTRAINT show_hall_theatre_movie_pk PRIMARY KEY ( showid,
                                                            hallid,
                                                            theatreid,
                                                            movieid );
```

Script Output

Task completed in 0.028 seconds

```
*Cause:    Self-evident.
*Action:   Remove the extra constraint.

Table HALL_THEATRE_MOVIE altered.


Table SHOW_HALL_THEATRE_MOVIE created.


Table SHOW_HALL_THEATRE_MOVIE altered.
```

9.11 Create Statement of Ticket_Customer Table

```
CREATE TABLE ticket_customer (
    ticketid   INTEGER NOT NULL,
    customerid INTEGER NOT NULL
);

ALTER TABLE ticket_customer ADD CONSTRAINT ticket_customer_pk PRIMARY KEY ( ticketid,
                                                                            customerid );

ALTER TABLE ticket_customer
    ADD CONSTRAINT ticket_customer_customer_fk FOREIGN KEY ( customerid )
```

Script Output

Task completed in 0.04 seconds

```
Table TICKET_CUSTOMER created.


Table TICKET_CUSTOMER altered.


Table TICKET_CUSTOMER altered.


Table TICKET_CUSTOMER altered.
```

9. Insert Statement

9.1 Insert Statement of Customer table

```
INSERT INTO customer (customerid, customername, customeraddress)
VALUES (1, 'Ram Sharma', 'Kathmandu');

INSERT INTO customer (customerid, customername, customeraddress)
VALUES (2, 'Sita Gurung', 'Pokhara');

INSERT INTO customer (customerid, customername, customeraddress)
VALUES (3, 'Hari Thapa', 'Lalitpur');

//Select Customer
SELECT * FROM customer;
```

Script Output × | Query Result ×

SQL | All Rows Fetched: 3 in 0.002 seconds

	CUSTOMERID	CUSTOMERNAME	CUSTOMERADDRESS
1	3	Hari Thapa	Lalitpur
2	1	Ram Sharma	Kathmandu
3	2	Sita Gurung	Pokhara

9.2 Insert Statement of Theatre table

```
//Insert Theatre table
INSERT INTO theatre (theatreid, theatrename, theatrecity)
VALUES (1, 'QFX Cinemas', 'Kathmandu');

INSERT INTO theatre (theatreid, theatrename, theatrecity)
VALUES (2, 'Big Movies', 'Pokhara');

INSERT INTO theatre (theatreid, theatrename, theatrecity)
VALUES (3, 'FCube Cinemas', 'Kathmandu');

INSERT INTO theatre (theatreid, theatrename, theatrecity)
VALUES (4, 'QFX Civil Mall', 'Kathmandu');
```

Script Output × | Query Result ×

Task completed in 0.035 seconds

```
Table TICKET_CUSTOMER altered.


Table TICKET_CUSTOMER altered.


1 row inserted.


1 row inserted.
```

9.3 Insert Statement of Movie table

```sql
//Insert Movie Table
INSERT INTO movie (movieid, movietitle, duration, language, genre, releasedate)
VALUES (1, 'Avengers Endgame', '3h 2m', 'English', 'Action', DATE '2019-04-26');

INSERT INTO movie (movieid, movietitle, duration, language, genre, releasedate)
VALUES (2, 'Avatar', '2h 42m', 'English', 'Sci-Fi', DATE '2009-12-18');

INSERT INTO movie (movieid, movietitle, duration, language, genre, releasedate)
VALUES (3, 'KGF Chapter 2', '2h 48m', 'Hindi', 'Action', DATE '2022-04-14');

INSERT INTO movie (movieid, movietitle, duration, language, genre, releasedate)
VALUES (4, 'Jawan', '2h 49m', 'Hindi', 'Action/Thriller', DATE '2023-09-07');
```

Script Output | Query Result

Task completed in 0.041 seconds

```
1 row inserted.

1 row inserted.

1 row inserted.

1 row inserted.
```

9.4 Insert Statement of Payment table

```sql
//Insert Payment Table
INSERT INTO payment (bookingdate, payment)
VALUES (DATE '2026-03-01', 'Cash');

INSERT INTO payment (bookingdate, payment)
VALUES (DATE '2026-03-02', 'Credit Card');

INSERT INTO payment (bookingdate, payment)
VALUES (DATE '2026-03-03', 'Debit Card');

INSERT INTO payment (bookingdate, payment)
VALUES (DATE '2026-03-04', 'Online Payment');
```

Script Output | Query Result

Task completed in 0.059 seconds

```
1 row inserted.

1 row inserted.

1 row inserted.

1 row inserted.
```

9.5 Insert Statement of Ticket table

```sql
//Insert Ticket Table
INSERT INTO ticket (ticketid, bookingdate, numberofseat)
VALUES (1, DATE '2026-03-01', 2);

INSERT INTO ticket (ticketid, bookingdate, numberofseat)
VALUES (2, DATE '2026-03-02', 4);

INSERT INTO ticket (ticketid, bookingdate, numberofseat)
VALUES (3, DATE '2026-03-03', 1);

INSERT INTO ticket (ticketid, bookingdate, numberofseat)
VALUES (4, DATE '2026-03-04', 3);
```

Script Output | Query Result

Task completed in 0.036 seconds

```
1 row inserted.

1 row inserted.

1 row inserted.

1 row inserted.
```

9.6 Insert Statement of Show table

```sql
//Insert Show Table
INSERT INTO show (showid, showdate, showtime)
VALUES (1, DATE '2026-03-10', '10:00 AM');

INSERT INTO show (showid, showdate, showtime)
VALUES (2, DATE '2026-03-10', '2:00 PM');

INSERT INTO show (showid, showdate, showtime)
VALUES (3, DATE '2026-03-10', '6:00 PM');
```

Script Output | Query Result

Task completed in 0.043 seconds

```
ORA-02291: integrity constraint (DATABASE.TICKET_PAYMENT_FK) violated - parent key not found

1 row inserted.

1 row inserted.

1 row inserted.
```

9.7 Insert Statement of Hall table

```
//Insert Hall Table
INSERT INTO hall (hallid, hallcapacity, hallname)
VALUES (1, '150', 'Hall A');

INSERT INTO hall (hallid, hallcapacity, hallname)
VALUES (2, '200', 'Hall B');

INSERT INTO hall (hallid, hallcapacity, hallname)
VALUES (3, '100', 'Hall C');
```

Script Output | Query Result

Task completed in 0.024 seconds

```
1 row inserted.

1 row inserted.

1 row inserted.

1 row inserted.
```

9.8 Insert Statement of Theatre_Movie table

```
//Insert THEATRE_MOVIE TABLE
INSERT INTO theatre_movie (movieid, theatreid)
VALUES (1,1);

INSERT INTO theatre_movie (movieid, theatreid)
VALUES (2,1);

INSERT INTO theatre_movie (movieid, theatreid)
VALUES (3,2)
```

Script Output | Query Result

Task completed in 0.031 seconds

```
1 row inserted.

1 row inserted.

1 row inserted.

1 row inserted.
```

9.9 Insert Statement of Hall_Theatre_Movie table

```
//Insert HALL_THEATRE_MOVIE TABLE
INSERT INTO hall_theatre_movie (hallid, theatreid, movieid)
VALUES (1,1,1);

INSERT INTO hall_theatre_movie (hallid, theatreid, movieid)
VALUES (2,1,2);

INSERT INTO hall_theatre_movie (hallid, theatreid, movieid)
VALUES (3,2,3);
```

Script Output | Query Result

Task completed in 0.047 seconds

```
*Cause:
*Action:

1 row inserted.


1 row inserted.


1 row inserted.
```

9.10 Insert Statement of Show_Hall_Theatre_Movie table

```
//Insert SHOW_HALL_THEATRE_MOVIE TABLE

INSERT INTO show_hall_theatre_movie (showid, hallid, theatreid, movieid)
VALUES (1,1,1,1);

INSERT INTO show_hall_theatre_movie (showid, hallid, theatreid, movieid)
VALUES (2,2,1,2);

INSERT INTO show_hall_theatre_movie (showid, hallid, theatreid, movieid)
VALUES (3,3,2,3);
```

Script Output | Query Result

Task completed in 0.052 seconds

```
1 row inserted.


1 row inserted.


1 row inserted.


1 row inserted.
```

9.11 Insert Statement of Ticket_Customer table

```
//Insert TICKET_CUSTOMER TABLE
INSERT INTO ticket_customer (ticketid, customerid)
VALUES (1,1);

INSERT INTO ticket_customer (ticketid, customerid)
VALUES (2,2);

INSERT INTO ticket_customer (ticketid, customerid)
VALUES (3,3);
```

Script Output × Query Result ×

SQL | All Rows Fetched: 3 in 0.012 seconds

	TICKETID	CUSTOMERID
1	1	1
2	2	2
3	3	3

10. Select Statement

10.1 Select Statement of Customer table

```
//Select Customer
SELECT * FROM customer;
```

Script Output × Query Result ×

SQL | All Rows Fetched: 1 in 0.11 seconds

	CUSTOMERID	CUSTOMERNAME	CUSTOMERADDRESS
1	3	Hari Thapa	Lalitpur

10.2 Select Statement of Theatre table

```
//Select Theatre
SELECT * FROM theatre;
```

ript Output × Query Result ×

SQL | All Rows Fetched: 5 in 0.003 seconds

	THEATREID	THEATRENAME	THEATRECITY
1	5	Midtown Galleria	Lalitpur
2	1	QFX Cinemas	Kathmandu
3	2	Big Movies	Pokhara
4	3	FCube Cinemas	Kathmandu
5	4	QFX Civil Mall	Kathmandu

10.3 Select Statement of Movie table

```
//Select Movie
SELECT * FROM movie;
```

Script Output × Query Result ×

SQL | All Rows Fetched: 4 in 0.007 seconds

	MOVIEID	MOVIETITLE	DURATION	LANGUAGE	GENRE	RELEASEDATE
1	1	Avengers Endgame	3h 2m	English	Action	26-APR-19
2	2	Avatar	2h 42m	English	Sci-Fi	18-DEC-09
3	3	KGF Chapter 2	2h 48m	Hindi	Action	14-APR-22
4	4	Jawan	2h 49m	Hindi	Action/Thriller	07-SEP-23

10.4 Select Statement of Payment table

```
//Select Payment Table
SELECT * FROM payment;
```

Script Output × Query Result ×

All Rows Fetched: 4 in 0.005 seconds

	BOOKINGDATE	PAYMENT
1	01-MAR-26	Cash
2	02-MAR-26	Credit Card
3	03-MAR-26	Debit Card
4	04-MAR-26	Online Payment

10.5 Select Statement of Ticket table

```
//Select Ticket table
SELECT * FROM payment;
```

Script Output × Query Result ×

All Rows Fetched: 4 in 0.003 seconds

	BOOKINGDATE	PAYMENT
1	01-MAR-26	Cash
2	02-MAR-26	Credit Card
3	03-MAR-26	Debit Card
4	04-MAR-26	Online Payment

10.6 Select Statement of Show table

```
//Select Show Table
SELECT * FROM show;
```

Script Output × Query Result ×

SQL | All Rows Fetched: 3 in 0.052 seconds

	SHOWID	SHOWDATE	SHOWTIME
1	1	10-MAR-26	10:00 AM
2	2	10-MAR-26	2:00 PM
3	3	10-MAR-26	6:00 PM

10.7 Select Statement of Hall table

```
//Select Hall Table
SELECT * FROM hall;
```

Script Output × Query Result ×

SQL | All Rows Fetched: 3 in 0.029 seconds

	HALLID	HALLCAPACITY	HALLNAME
1	1	150	Hall A
2	2	200	Hall B
3	3	100	Hall C

10.8 Select Statement of Theatre_Movie table

```
//Select THEATRE_MOVIE TABLE
SELECT * FROM theatre_movie;
```

Script Output | Query Result

All Rows Fetched: 3 in 0.002 seconds

	MOVIEID	THEATREID
1	1	1
2	2	1
3	3	2

10.9 Select Statement of Hall_Theatre_Movie table

```
//Select HALL_THEATRE_MOVIE TABLE
SELECT * FROM hall_theatre_movie;
```

Script Output | Query Result

All Rows Fetched: 3 in 0.022 seconds

	HALLID	THEATREID	MOVIEID
1	1	1	1
2	2	1	2
3	3	2	3

10.10 Select Statement of Show_Hall_Theatre_Movie table

```
//Select SHOW_HALL_THEATRE_MOVIE TABLE
SELECT * FROM show_hall_theatre_movie;
```

Script Output | Query Result

All Rows Fetched: 3 in 0.014 seconds

	SHOWID	HALLID	THEATREID	MOVIEID
1	1	1	1	1
2	2	2	1	2
3	3	3	2	3

10.11 Select Statement of Ticket_Customer table

```
//Select  TICKET_CUSTOMER TABLE
SELECT * FROM ticket_customer;
```

Script Output | Query Result

All Rows Fetched: 3 in 0.012 seconds

	TICKETID	CUSTOMERID
1	1	1
2	2	2
3	3	3

11. Forms

11.1 Dashboard page

11.2 Complex form and queries

11.2.1 SQL Queries

11.2.1.1 User Ticket Details

```
SELECT
        c.CUSTOMERID,
        c.CUSTOMERNAME,
        c.CUSTOMERADDRESS,
        t.TICKETID,
        t.NUMBEROFSEAT,
        t.BOOKINGDATE,
        p.PAYMENT
    FROM
        CUSTOMER c,
        TICKET_CUSTOMER tc,
        TICKET t,
        PAYMENT p
```

```
WHERE
    c.CUSTOMERID = tc.CUSTOMERID
    AND tc.TICKETID = t.TICKETID
    AND t.BOOKINGDATE = p.BOOKINGDATE
    AND c.CUSTOMERID = :CUSTOMERID
    AND t.BOOKINGDATE >= ADD_MONTHS(SYSDATE, -6)
    AND t.BOOKINGDATE <= SYSDATE
ORDER BY
    t.BOOKINGDATE DESC;
```

11.2.1.2 TheaterCityHall MovieC

```
SELECT
    t.THEATREID, t.THEATRENAME, t.THEATRECITY,
    h.HALLID, h.HALLNAME, h.HALLCAPACITY,
    m.MOVIEID, m.MOVIETITLE, m.GENRE, m.LANGUAGE, m.DURATION,
    s.SHOWID, s.SHOWTIME, s.SHOWDATE
FROM
    THEATRE t, HALL h, MOVIE m, SHOW s,
    THEATRE_MOVIE tm, HALL_THEATRE_MOVIE htm,
    SHOW_HALL_THEATRE_MOVIE shtm
WHERE
    t.THEATREID = tm.THEATREID
    AND t.THEATREID = :THEATREID
    AND tm.MOVIEID = m.MOVIEID
    AND htm.THEATREID = t.THEATREID
```

```
        AND htm.MOVIEID = m.MOVIEID

        AND htm.HALLID = h.HALLID

        AND shtm.SHOWID = s.SHOWID

        AND shtm.THEATREID = t.THEATREID

        AND shtm.MOVIEID = m.MOVIEID

        AND shtm.HALLID = h.HALLID

    ORDER BY

        s.SHOWDATE, s.SHOWTIME;
```

11.2.1.2 MovieTheatherCityHallOccupancyPerformerC

```
SELECT * FROM (
    SELECT
        m.MOVIETITLE,
        t.THEATRENAME,
        t.THEATRECITY,
        h.HALLNAME,
        h.HALLCAPACITY,
        SUM(tk.NUMBEROFSEAT) AS TOTAL_SEATS_BOOKED,
        ROUND((SUM(tk.NUMBEROFSEAT) / h.HALLCAPACITY) * 100, 2) AS
OCCUPANCY_PERCENTAGE
    FROM
        MOVIE m,
        THEATRE t,
        HALL h,
        SHOW s,
        THEATRE_MOVIE tm,
        HALL_THEATRE_MOVIE htm,
        SHOW_HALL_THEATRE_MOVIE shtm,
        TICKET tk,
        TICKET_CUSTOMER tcu,
        PAYMENT p
    WHERE
        t.THEATREID = tm.THEATREID
        AND tm.MOVIEID = m.MOVIEID
        AND htm.THEATREID = t.THEATREID
```

```
    AND htm.MOVIEID = m.MOVIEID
    AND htm.HALLID = h.HALLID
    AND shtm.SHOWID = s.SHOWID
    AND shtm.THEATREID = t.THEATREID
    AND shtm.MOVIEID = m.MOVIEID
    AND shtm.HALLID = h.HALLID
    AND tcu.TICKETID = tk.TICKETID
    AND tk.BOOKINGDATE = p.BOOKINGDATE
    AND p.PAYMENT IN ('Cash','Credit Card','Debit Card','Online Payment')
    AND m.MOVIEID = :MOVIEID
GROUP BY
    m.MOVIETITLE,
    t.THEATRENAME,
    t.THEATRECITY,
    h.HALLNAME,
    h.HALLCAPACITY
ORDER BY OCCUPANCY_PERCENTAGE DESC)
WHERE ROWNUM <= 3";
```

11.2.2 Complex Form

11.2.2.1 Complex form of User Ticket Details

User Ticket Details (Last 6 Months)

Select Customer: Hari Thapa

Customer ID	Name	Address	Ticket ID	Seats	Booking Date	Payment Method
3	Hari Thapa	Lalitpur	3	1	3/3/2026 12:00:00 AM	Debit Card

11.2.2.2 Complex form of TheaterCityHall MovieC

Top 3 Theatre Hall Occupancy by Movie

Select Movie: Avatar

MOVIETITLE	THEATRENAME	THEATRECITY	HALLNAME	HALLCAPACITY	TOTAL_SEATS_BOOKED	OCCUPANCY_PERCENTAGE
Avatar	QFX Cinemas	Kathmandu	Hall B	200	7	3.5

11.2.2.3 Complex form of MovieTheatherCityHallOccupancyPerformerC

Top 3 Theatre Hall Occupancy by Movie

Select Movie: Avengers Endgame

MOVIETITLE	THEATRENAME	THEATRECITY	HALLNAME	HALLCAPACITY	TOTAL_SEATS_BOOKED	OCCUPANCY_PERCENTAGE
Avengers Endgame	QFX Cinemas	Kathmandu	Hall A	150	7	4.67

11.3 Simple Form

11.3.1 Customer.aspx

	Customer ID	Customer Name	Address
Edit Delete	1	Ram Sharma	Kathmandu
Edit Delete	2	Sita Gurung	Pokhara
Edit Delete	3	Hari Thapa	Lalitpur

Add New Customer

Customer ID:

Customer Name:

Address:

Add Customer

11.3.2Movie.aspx

	MOVIEID	MOVIETITLE	DURATION	GENRE	LANGUAGE	RELEASEDATE
Edit Delete	1	Avengers Endgame	3h 2m	Action	English	4/26/2019 12:00:00 AM
Edit Delete	2	Avatar	2h 42m	Sci-Fi	English	12/18/2009 12:00:00 AM
Edit Delete	3	KGF Chapter 2	2h 48m	Action	Hindi	4/14/2022 12:00:00 AM
Edit Delete	4	Jawan	2h 49m	Action/Thriller	Hindi	9/7/2023 12:00:00 AM

Add

11.3.3 Theatre.aspx

	THEATREID	THEATRENAME	THEATRECITY
Edit Delete	5	Midtown Galleria	Lalitpur
Edit Delete	1	QFX Cinemas	Kathmandu
Edit Delete	2	Big Movies	Pokhara
Edit Delete	3	FCube Cinemas	Kathmandu
Edit Delete	4	QFX Civil Mall	Kathmandu

Add

11.3.4 Hall.aspx

	HALLID	HALLCAPACITY	HALLNAME
Edit Delete	1	150	Hall A
Edit Delete	2	200	Hall B
Edit Delete	3	100	Hall C

Add

11.3.5 Show.aspx

	SHOWID	SHOWTIME	SHOWDATE
Edit Delete	1	10:00 AM	3/10/2026 12:00:00 AM
Edit Delete	2	2:00 PM	3/10/2026 12:00:00 AM
Edit Delete	3	6:00 PM	3/10/2026 12:00:00 AM
Edit Delete	4	3:00 PM	4/10/2026 12:00:00 AM

Add

11.3.6 Ticket.aspx

	TICKETID	BOOKINGDATE	NUMBEROFSEAT
Edit Delete	1	3/1/2026 12:00:00 AM	2
Edit Delete	2	3/2/2026 12:00:00 AM	4
Edit Delete	3	3/3/2026 12:00:00 AM	1
Edit Delete	4	3/4/2026 12:00:00 AM	3

Add

11.3.7 Payment.aspx

Add

	BOOKINGDATE	PAYMENT
Edit Delete	3/1/2026 12:00:00 AM	Cash
Edit Delete	3/2/2026 12:00:00 AM	Credit Card
Edit Delete	3/3/2026 12:00:00 AM	Debit Card
Edit Delete	3/4/2026 12:00:00 AM	Online Payment

15. Conclusion

To conclude, the coursework was directed towards designing and developing a web based database system to a movie theatre, Kumari Cinemas, to enhance the process of booking movie tickets. The first part of the project involved an analysis of the case study and determination of the system requirements, business rules and relations between various entities (customers, movies, theatres, halls, shows and tickets etc.). The process of normalization was done between UNF and to 3NF in order to remove redundancy and to provide data integrity by establishing primary keys, foreign keys, and functional dependencies.

Entity Relationship Diagram (ERD) was developed using Oracle SQL Developer Data Modeler based on the normalized relations to show the structure of the database and the relationship between entities. The last schema was introduced in Oracle SQL developer with the help of DDL scripts to create tables whereas INSERT and SELECT statement was utilized to fill and test the data. Also a web application has been created in ASP.NET with C# which comprised of simple data management forms and complex forms to make more complicated queries like ticket details of users, movie show details in theatres and the seat occupancy analysis.

Comprehensively, the designed system offers an organized and effective mechanism of controlling the movie tickets booking and cinema facilities. It assists in raising the efficiency of operations, minimizing the number of errors that can be made manually, and improving customer experience as it allows to better manage the movie schedule, halls, and sell the tickets with the help of a centralized database system.